DAAR
Devoir 1 - egrep

Camille DIVISIA

Introduction

La commande UNIX grep (Global Regular Expression Print) recherche, ligne par ligne, les lignes d'un fichier qui correspondent à une expression régulière (RegEx). Sa variante egrep (extended grep) utilise les expressions régulières étendues (Extended Regular Expressions, ERE) : les métacaractères y sont interprétés directement, sans avoir à être échappés comme avec grep. Formellement, une expression régulière est la formalisation d'un langage algébrique reconnaissable par un automate fini déterministe (DFA), au sens de la théorie des langages formels. Cette équivalence est à la base de la plupart des implémentations de egrep. Étant donné un fichier texte de l lignes et une RegEx, il s'agit de retrouver les lignes contenant une correspondance. Le problème se décompose en l recherches indépendantes de la RegEx, une par ligne. Ce projet vise à comprendre le fonctionnement de egrep en en réalisant un clone en Python, nommé egrep.py, à partir des algorithmes vus en cours. Son résultat doit être identique à celui de la commande UNIX egrep, qu'on utilise comme référence de correction et de performance.

Définition du problème et structures de données

1.1 Définition

Pour un fichier de *l* lignes, egrep affiche chaque ligne qui contient une suite de caractères correspondant au langage dénoté par l'expression régulière. Par ordre de priorité, la grammaire reconnue par notre analyseur est la suivante:

- Alternation (" | "), priorité la plus faible
- Concaténation (juxtaposition de caractères, pas de symboles)
- Les symboles *, +, ?
- Les lettres, les points et les parenthèses, priorité la plus forte

Certaines conventions de GNU egrep sont reproduites volontairement pour obtenir les mêmes sorties :

- Le point ne reconnaît pas le saut de ligne
- Un quantificateur sans opérande (*a) est ignoré
- Une parenthèse fermante sans ouvrante est un littéral
- Le motif vide "" accepte toutes les lignes.
- Le programme accepte aussi l' option –stats pour montrer le nombre d'états

1.2 Chaîne de traitement

Le traitement suit les cinq étapes vues en cours avec sa propre structure de données.

Étape 1 : L'arbre syntaxique

L'objectif de cette étape est de convertir l'expression régulière en un arbre syntaxique, en respectant les priorités et l'association à gauche pour les opérations " | " et "." comme montré en Figure 1.

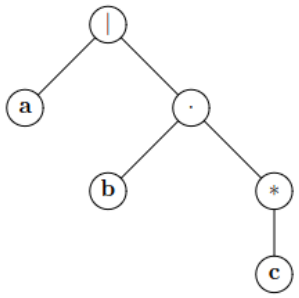


Figure 1. Arbre syntaxique du RegEx a | bc* (Aho-Ullman Fig.10.29)

Étape 2 : NDFA avec ε-transitions

A partir de l'arbre de syntaxe, on construit l'automate fini. Chaque état du NDFA possède un identifiant, un dictionnaire de transitions (symbole → liste d'états) et une liste de transitions epsilon. La construction est récursive sur l'arbre. Chaque fragment a un unique état initial et un unique état final. L'état initial n'a aucun arc entrant et l'état final aucun arc sortant, comme dans la preuve du chapitre 10 d'Aho-Ullman. Chaque feuille, chaque alternative et chaque *, + ou ? crée exactement deux états. La concaténation en crée zéro : elle relie les deux fragments par une ε-transition.

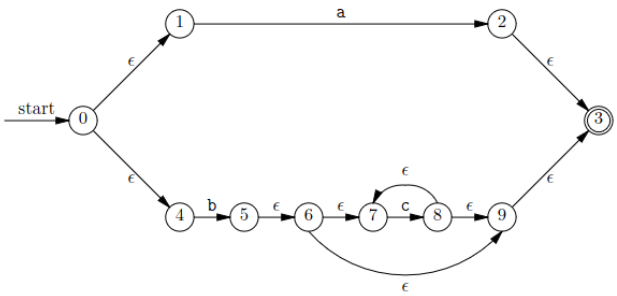


Figure 2. Automate fini avec ε-transitions du RegEx a | bc*(Aho-Ullmann Fig. 10.26)

Étape 3 : DFA.

Pour construire le DFA à partir du NDFA, on utilise la méthode des sous-ensembles et en éliminant les ε-transitions. Chaque état du DFA est l'ensemble des états du NDFA atteints après lecture d'un mot. La correction se démontre par induction sur la longueur du mot : l'état atteint par le DFA est exactement l'ensemble des états atteignables par le NDFA avec ce mot. Les sous-ensembles sont découverts en largeur à partir de la fermeture de l'état initial. Un état du DFA est acceptant s'il contient l'état final du NDFA.

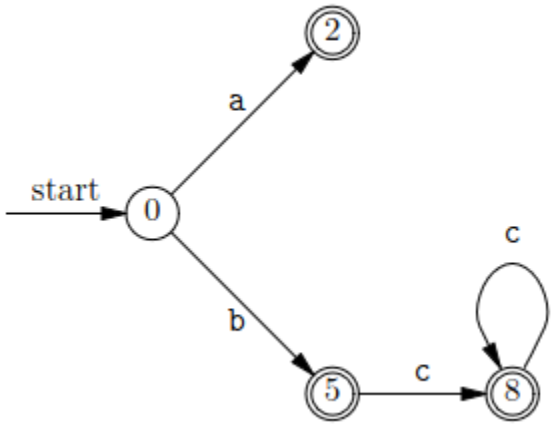


Figure 3. DFA du RegEx a | bc* (Aho-Ullman Fig. 10.35)

Étape 4 : Minimisation.

Après avoir construit le DFA, on cherche à simplifier l'automate. On applique l'algorithme de Moore:

- La partition initiale sépare états acceptants et non acceptants
- A chaque tour, on regroupe les états d'une classe par signature, c'est-à-dire le vecteur des classes d'arrivée pour chaque symbole
- on ne s'arrête quand plus aucune classe est divisible.

La minimisation de Moore repose sur la notion d'états équivalents: deux états sont distinguables s'ils diffèrent par leur caractère acceptant, ou si une même lettre les envoie vers des états distinguables. Il existe un unique DFA minimal pour un langage donné. Les états du NDFA peuvent tous atteindre l'état final, car la grammaire n'a pas d'opérande ∅. Le seul état mort possible est donc l'ensemble vide.

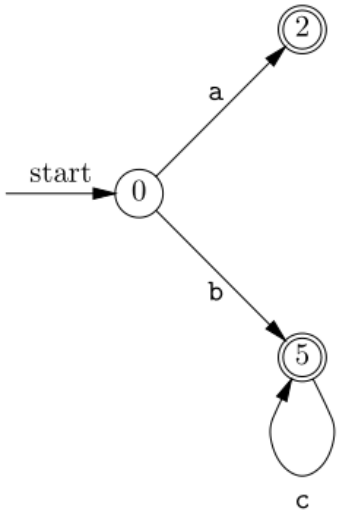


Figure 4. Simplification de l'automate pour reconnaître le RegEx a | bc* (Aho-Ullman Fig. 10.36)

Étape 5 : Reconnaissance - Pattern Matching

Enfin, pour faire du pattern matching, on a implémenté l'algorithme KMP. Lorsque la RegEx est une simple concaténation de lettres, le problème se réduit à une recherche de sous-chaîne. KMP évite ces retours en arrière grâce à un précalcul sur le facteur : la table "Retenue" ou "CarryOver", qui donne, pour chaque position j dans F, la longueur du plus long bord propre de F[0..j−1], c'est-à-dire du plus long préfixe de F qui est aussi son suffixe. Par exemple, [m,a,m,a,m,i,a] donne [-1,0,-1,0,-1,3,0,0]. Lors d'un désaccord en position j du facteur, on ne relit pas le texte : on reprend la comparaison à la position Retenue[j] du facteur, la lecture du texte continuant sur le même caractère (ou avançant d'un cran si Retenue[j] vaut −1). Le texte n'est ainsi parcouru qu'une fois.

1.3 Arguments critiques

La construction par sous-ensembles peut donner 2^n états, ce qui crée une explosion du nombre d'états. L'exemple classique est le pattern "(a|b)*a(a|b)…(a|b)" avec n−1 facteurs (a | b) finaux : son DFA minimal a 2^n états, car il doit mémoriser les n dernières lettres lues.

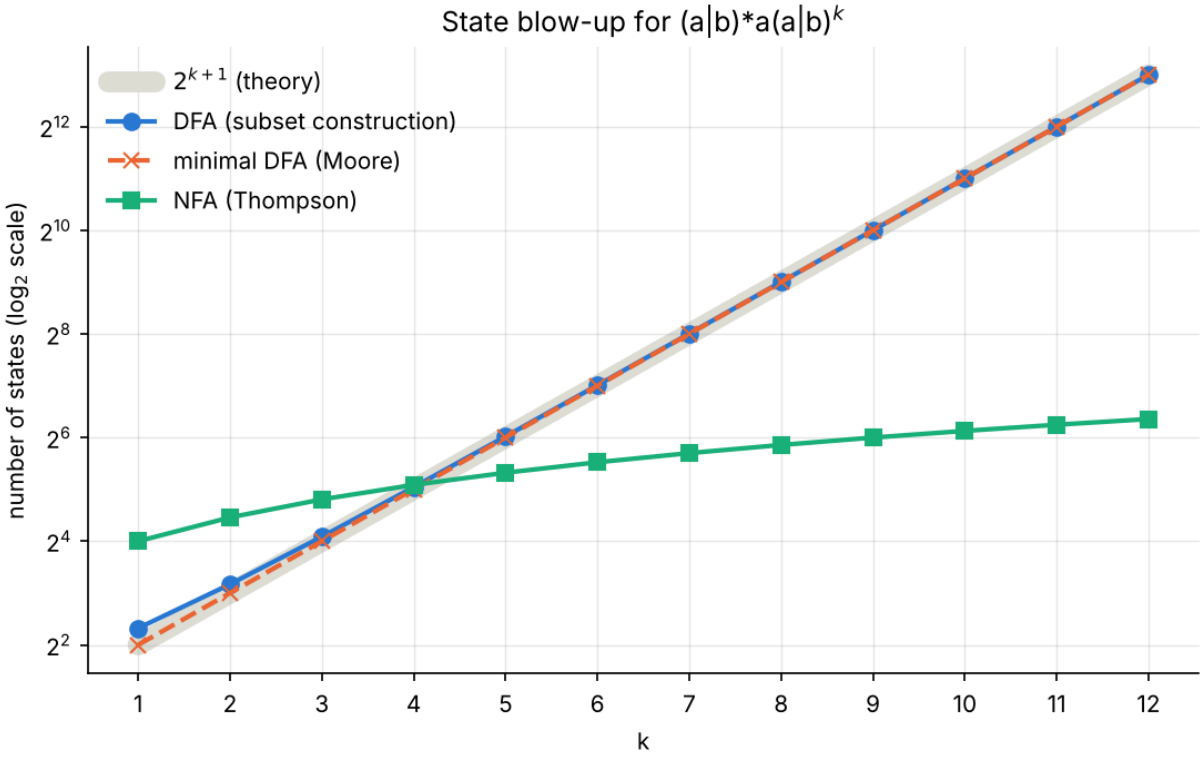


Figure 5. Graphe montrant l'explosion d'états sur le RegEx (a|b)*a(a|b)^k

Le DFA est construit avant la première lecture. Cette manière de construire le DFA est appelée aussi "construction eager". Le coût de préparation est maximal, mais la lecture n'est qu'une suite d'accès à une table. Cela permet aussi de minimiser, car Moore suppose un automate complet. La minimisation s'exécute une seule fois par motif.

3. Tests et performances

Comparaison des résultats

Avant de tester les vitesses d'exécution de GNU egrep et egrep.py, on a testé la sortie pour la recherche de chaînes de caractères correspondant au motif "S(a|g|r)+on" dans le fichier textuel https://www.gutenberg.org/files/56667/56667-0.txt, correspondant au livre "A History of Babylon, From the Foundation of the Monarchy to the Persian Conquest History of Babylonia vol. 2." par Leonard William King. On remarque la similarité des résultats en sortie pour GNU egrep et egrep.py dans la Figure 6.

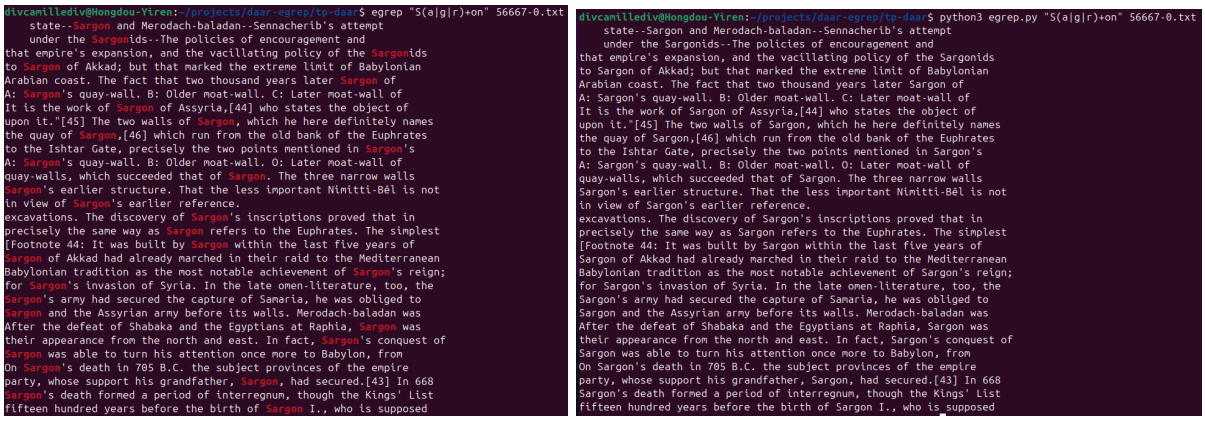


Figure 6: Motif “S(a|g|r)+on” sur GNU egrep (à gauche) et egrep.py (à droite)

Vitesses de performances

Avec le script bench_egrep.py, on compare les vitesses de performances entre GNU egrep et egrep.py. Il s’avère que le temps de recherche croît linéairement avec la taille du texte. Par contre, le graphe montre aussi un écart entre la vitesse d’exécution de GNU egrep et egrep.py d’après la Figure 7. Le pattern matching avec KMP permet cependant d’augmenter légèrement la vitesse d’exécution quand il s’agit de chaînes de lettres ASCII. On a aussi comparé la vitesse d’exécution selon les patterns.

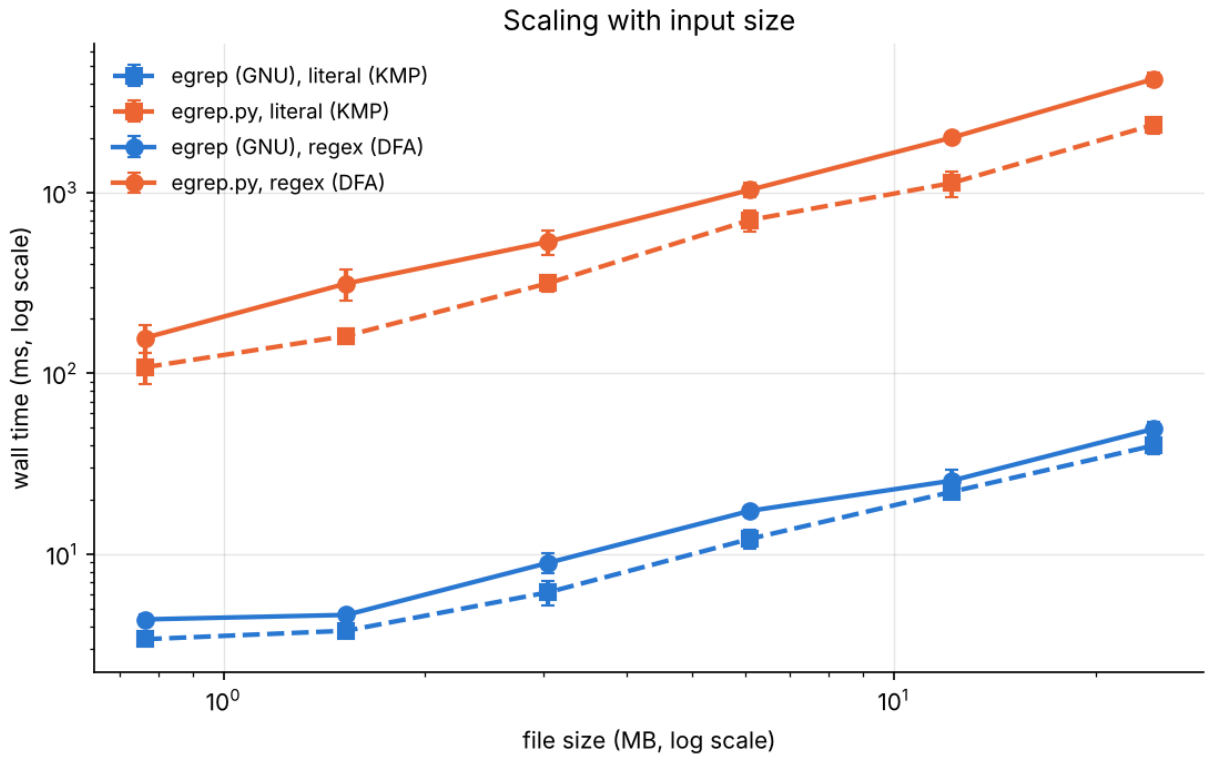


Figure 7. Temps d’exécution de egrep.py vs GNU egrep selon la taille du fichier

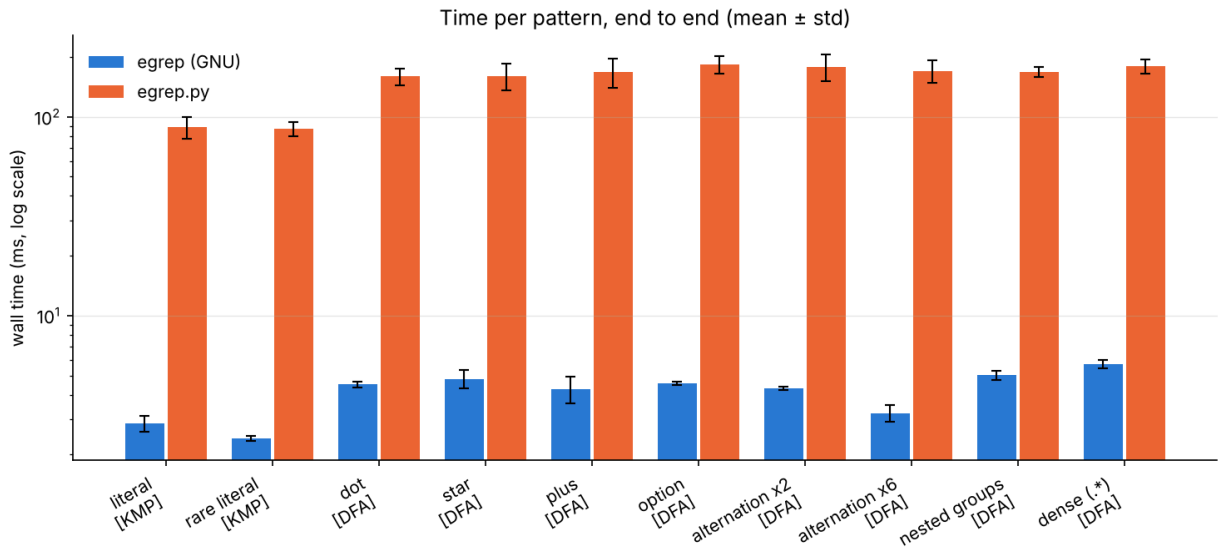


Figure 8: Temps d'exécution par motif

On a aussi mesuré le temps que prend chaque partie de egrep.py, c'est à dire le temps d'interprétation de Python, la construction de l'automate et le pattern-matching pour chaque motif testé auparavant.

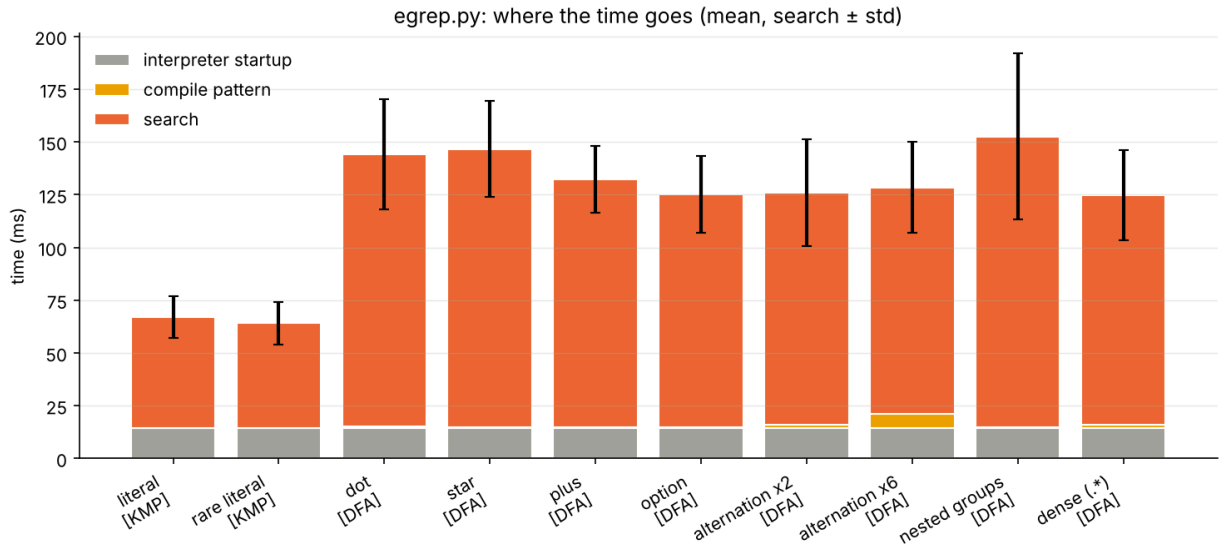


Figure 9: Répartition du temps d'exécution d'egrep.py

Limites

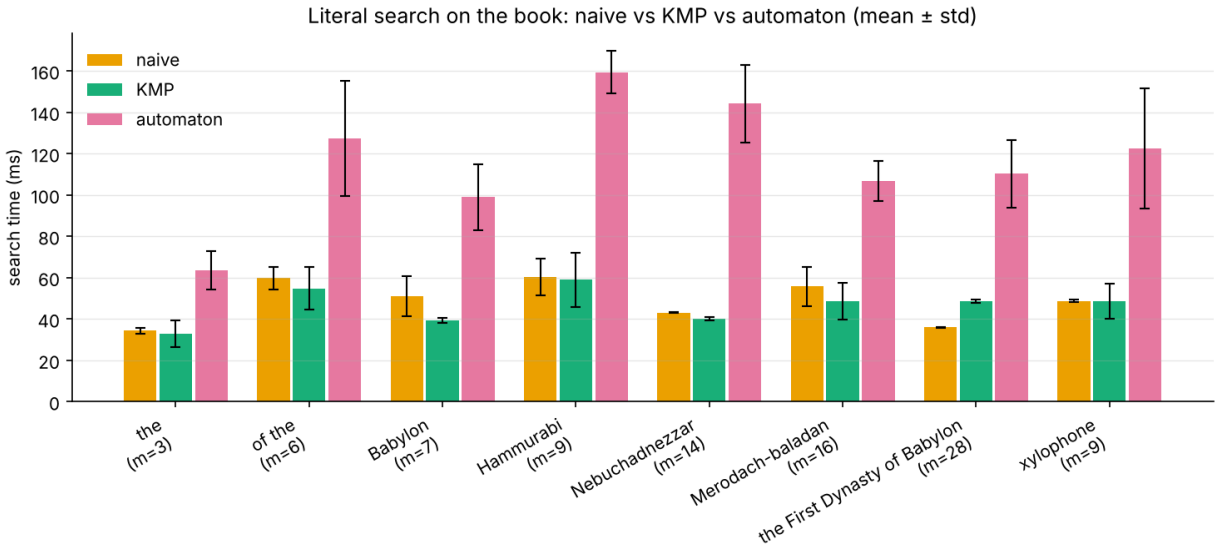


Figure 10. Temps de recherche pour différentes chaînes de lettres ASCII, recherche naive vs KMP vs automate

La décomposition du temps montre que la phase de recherche est la plus coûteuse. On a implémenté KMP comme stratégie dédiée aux motifs qui sont de simples concaténations de lettres. Les autres motifs passent par la chaîne NFA → DFA → Minimisation, qui sert de repli. KMP garantit un temps linéaire et convient bien aux petits alphabets, comme démontré dans la Figure 10.

GNU grep reste nettement plus rapide, pour deux raisons. La première est le langage : il s'agit de C compilé, contre une boucle Python qui exécute plusieurs instructions par caractère. La seconde, qu'on formule comme une hypothèse non vérifiée, est l'algorithme : GNU grep extrait de la RegEx des chaînes fixes, les cherche avec un algorithme à sauts de type Boyer-Moore, puis n'exécute l'automate que sur les lignes candidates. Notre KMP et notre DFA lisent chaque caractère, sans jamais sauter.

Conclusion et perspectives

On a réalisé un clone de egrep en Python, appelé egrep.py, qui suit la méthode vu en cours: arbre syntaxique, NDFA, DFA, minimisation de Moore, puis reconnaissance ligne par ligne. Côté performances, la lecture croît linéairement avec la taille du texte. En revanche, egrep.py est plus lent que GNU grep. Ceci a été un premier essai de reproduire la commande egrep pour mieux comprendre son fonctionnement théoriquement. Il est toujours possible d'optimiser et se rapprocher davantage de GNU egrep. Une perspective envisagée serait de simuler directement le NDFA (construction lazy opposé à notre implémentation en construction eager) en suivant l'ensemble des états actifs par caractère, ou en ne calculant les états du DFA qu'à la demande avec un cache. Les deux évitent l'explosion d'états quand le texte n'en visite qu'une petite partie. Une autre perspective serait de vérifier l'hypothèse des chaînes fixes et de la recherche avec un algorithme à sauts.

Bibliographie

- A. V. Aho, J. D. Ullman, *Foundations of Computer Science*, chap. 10 « Patterns, Automata, and Regular Expressions »